EPFL - Computer Science

COM-407 Advanced Networks

Lecture 06 — November 27, 2025

Author: Jonathan Dutoit

Professor: Pavlos Nikoupoulos

Transport Layer (Part 1)

TCP and UDP

(MAC Continued) 6. Security Aspects

- MAC addresses are sent in cleartext
- Many packets are broadcast (especially in shared-medium LANs)

Possible attacks:

- Eavesdropping
- Free riding
- Impersonation

Solutions

- Access control: requires uses to show credentials before allowing access to the network;
  - shared = user knows a password, same for all (e.g., WiFI WPA-Personal)
  - per-user = user has a personal password (e.g., WiFi WPA-Enterprise
  - **prevents free riding**
- Authenticity: we can use cryptography to authenticate src MAC address
  - **prevents impersonation**, easier if access control is per use
- Confidentiality: we can use cryptography to encrypt MAC frame payloads (but not MAC addresses)
  - **prevents eavesdropping**

At home:

- WiFi WPA-Personal provides access control but does not authenticate individual users
- Impersonation is possible by whoever knows the shared WiFI password

EPFL:

- WiFi uses per-user access control, authentication and encryption
- Eavesdropping, Free-riding and impersonation are no longer possible

Securing the MAC layer

IEEE 802.1X for access control

IEEE 802.1AE (MACSEC) for authentication and encryption

steps:
i. Host negotiates access with WiFI base station (BS) or Ethernet switch
ii. BS/Switch contacts an authentication server (e.g., RADIUS)
iii. Host and server establish a secure (TLS) connection and run an inner authentication protocol (e.g., PEAP/MSCHAPv2), where host "shows" credentials and session keys are commonly derived
iv. If host is accepted, frames between host and BS/Switch are authenticated and encrypted using session keys

- authentication covers MAC payload and src MAC address
- encryption covers MAC payload

Conclusion

- The MAC layer fro wireless medium (WiFI)
  takes care of **sharing** the radio waves in a local environment
- The MAC layer for wired medium (Ethernet)

- 3

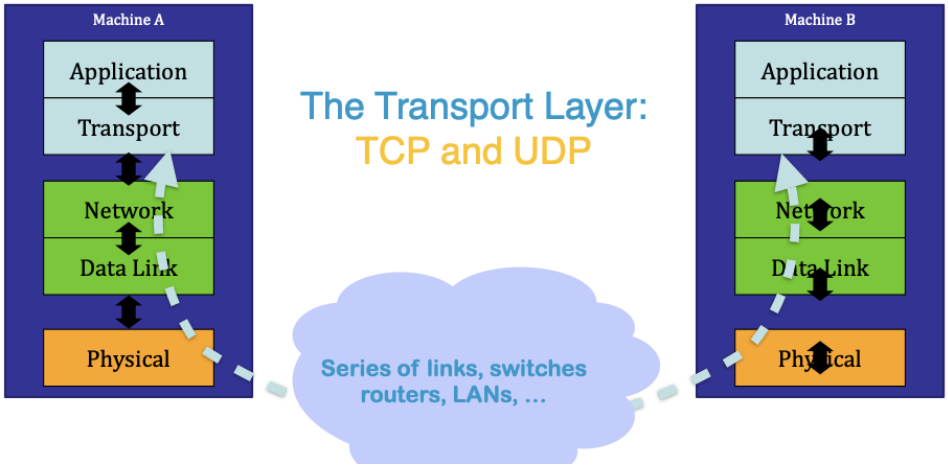
We saw 2 mechanisms to interconnect devices:

  - the network layer (with IP router), and
  - the MAC layer (with switches)

What are the differences?

- Switches are plug-and-play devices, independent from IP numbering plans and whether IPv4 or IPv6 is used (as opposed to router).
- but use forwarding tables that are not structured (no aggregation by prefix).
  - For every received frame, a switch must search the entire table, because of the exact match.
  - If network becomes too large, table size / lookup times may become impractical.
    * Prefer router for large networks
- Historically, routers were in software and switches in hardware; nowadays, every combination exists.

Transport Layer



Contents

1. UDP
2. TCP Basics: Sliding Window and Flow Control
3. TCP Connections and Sockets
4. More TCP Bells and Whistles
5. Secure Transport
6. Where should packet losses be repaired ?

Reminder from 1st lecture: Transport layer services

- Network + Data link + Physical layer carry packets end-toend
- packets may be lost due to:
  - errors at the physical layer
  - buffer-overflow events at router-s/switches
- or reordered due to:
  - following different paths to destination

- Transport layer
  - makes networks services available to pgraoms
  - is in end-systems only, not in routers' data plane (i.e. not at forwarding level)
  - may handle packet loss/reordering or not
    - UDP not reliable transfer, takes no action
    - TCP reliable, in-order transfer by using sophisticated mechanisms

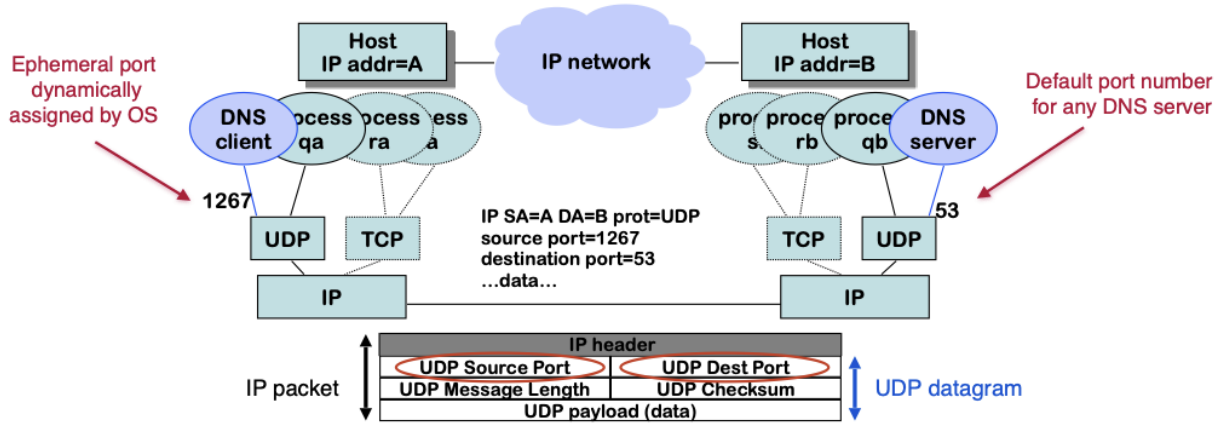
What is the definition of a "server" in networks?

A. A machine that hosts resources to be used in the web
B. A computer with high CPU performance
C. A computer with large data storage
D. The role of a program
E. The role of a program that allows users to access large amount of resources
F. None of the above
G. I don't know

Answer: D.
Formally, a server is a role at the transport layer, where the program waits for requests to come. In contrast, a client initiates communication to a server. **Port forwarding** comes from the need of allowing the client to start the connection.

Reminder from 1st lecture: Port Numbers

- Assigned by OS to identify processes within a host
- servers' port numbers must be well-known to clients (e.g., 53 for DNS, 80 for HTTP, 443 for HTTPS)
- src and dest port numbers are inside transport-layer header



1. UDP: message-oriented and unreliable

- UDP delivers either the exact message (a.k.a. "datagram") or nothing
- Each messages $\leq$ 65535 bytes (32 bits sequences)
- If a message is too large to fit into a single IP packet (i.e. larger than MTU), then IP layer fragments it:
  - not visible to the transport layer
  - at source's IP layer, metadata about fragments is added to IP header

  - if a fragment/piece is lost, then the entire message is lost

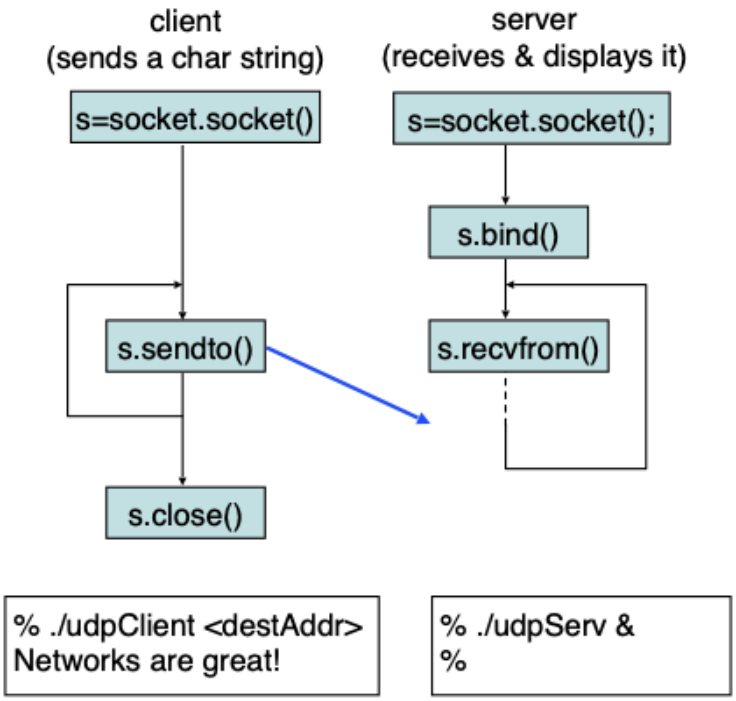
- Application layer must handle out of order or lost messages, if needed
  - e.g., DNS client resends DNS query if no response is received

How is UDP implement in practice?

Via sockets = programming interfaces

  - (in Unix) for read/write: sockets ≈ files

- `socket(socket.AF_INET, ...)` resp. `socket(socket.AF_INET6, ...)` creates and IPv4 (resp. IPv6) socket, returns an ID (= file descriptor)
- bind() associates an IP address + port number with a socket, can be skipped for the client
  - Port = 0 *Rightarrow* any available port
  - IP address = 0 ⇒ all host's IP addresses
- sendto() specifies: dest IP addresses, dest port number, and the message to send
- recvfrom() blocks until a message is received for this port number; it returns: src IP address, src port number and the message.



What socket to open? IPv4 or IPv6?

- Transport layer is not affected by the choice of IP (no UDPv6, not TCPv6)

- But, an application must choose IPv4 or IPv6, or better support both

How? Use `socker.getaddrinfo()` to let the DNS give you whatever is available

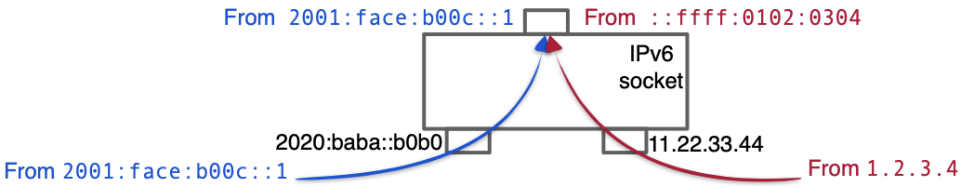
IPv6 sockets can be dual-stack

= bound to both IPv6 and IPv4 addresses of the local host

= receive packets from IPv6 and/or IPv4 correspondents

How? The correspondents' IPv4 addresses are mapped to IPv6 addresses

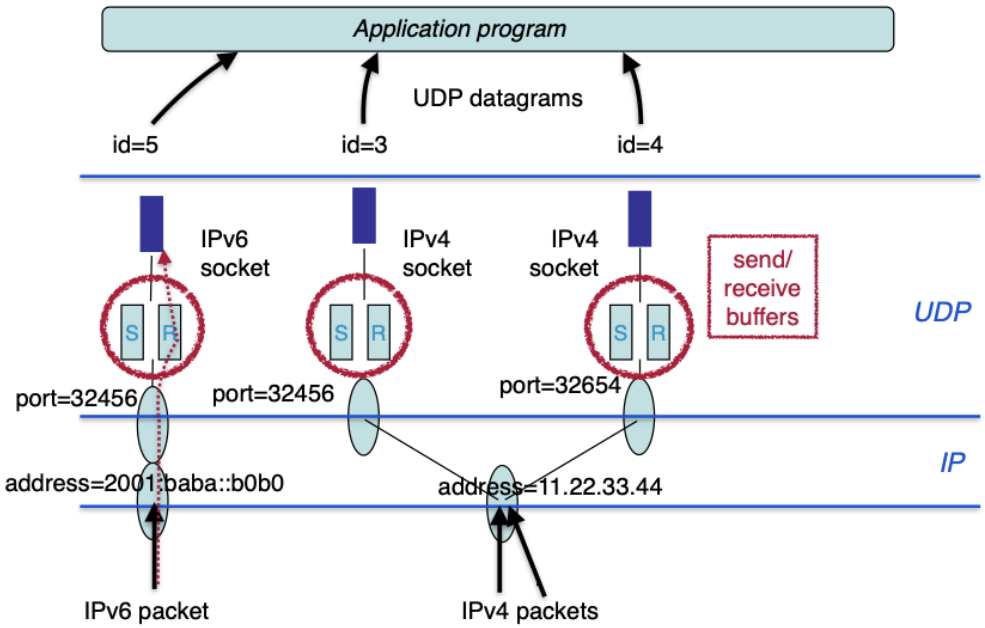
- using the IPv4-mapped IPv5 address format
- i.e., by appending the IPv4 address to prefix `::ffff:0:0/96`



- Default in Linux, must be enabled for every socket (with `setsockopt` in Windows.
- An IPv4 socket cannot be dual-stack. Why?
  - That's because the IPv6 address space is munch longer in bits than the one from IPv4. Hence, an IPv4 socket cannot receive data from an IPv6 source address.

How does the Operating System view UDP?

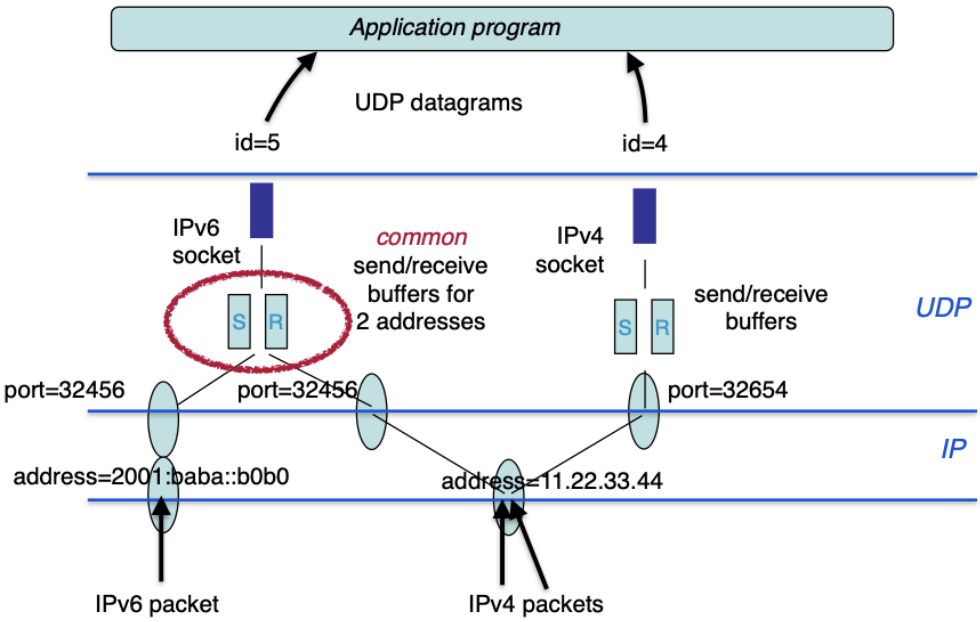
UDP datagrams are delivered to sockets based on *dest IP address* and *port number*.



Note: buffers can overflow! Receiver buffer is needed to assemble messages, sender buffer is necessary for congestion (for example WiFI carrier waits the channel to be free)

With a dual-stack IPv6 socket?

Socket 5 is bound to any local address, which includes IPv6 and IPv4 addresses, and to port 32456



User's browser sends DNS query to DNS server, over UDP. What happens if query or answer is lost?

A. Name resolver in browser waits for timeout, if no answer received before timeout, sends again
B. Messages cannot be lost because UDP assures message integrity
C. UDP detects the loss and retransmits
D. I don't know

Answer: A

Recap - UDP

On the sending side: OS sends the UDP messages ASAP, but also uses a buffer to store data if interface is busy. OS may algo fragment the message if needed.

On the receiving side: OS re-assembles IP fragments of UDP message (if needed) and keeps the message in received buffer ready to be read. The message is:

- "*consumed*" when application reads
- or "*dropped*" because of an overflow event

A socket is bound to a single port and one or multiple IP addresses of the local host

2. TCP offers reliable, in-order delivery

What?

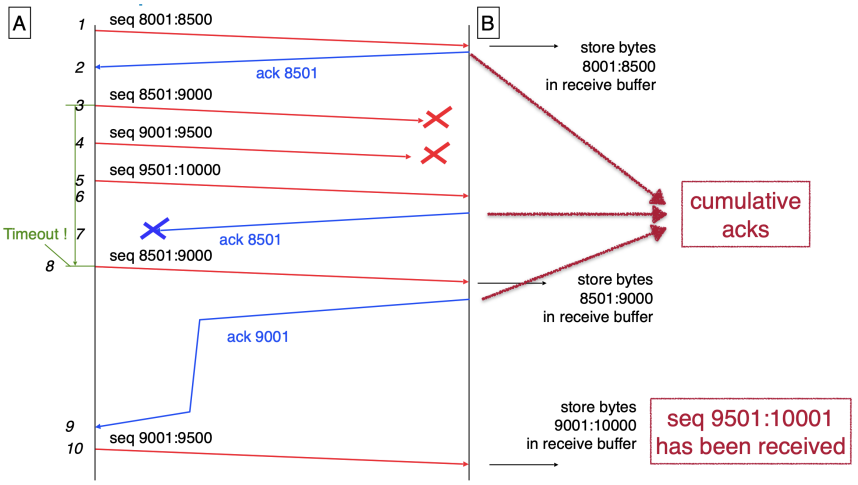
- TCP guarantees that all data is delivered in order and without loss, unless the connection is broken

How?

- Detects reordering and loss, via sophisticated mechanisms:
  - per-byte sequences numbers
  - a connection-setup phase for the sender/receiver to synchronize their sequence numbs
  - acknowledgements; if loss is detected, TCP retransmits
- further optimizations, e.g.:
  - flow control avoids buffer at the receiver
  - TCP knows the allowable maximum segment size (MSS) and segments data accordingly ⇒ avoids fragmentation at the IP layer

TCP Basic Operation 1: SEQ and ACK

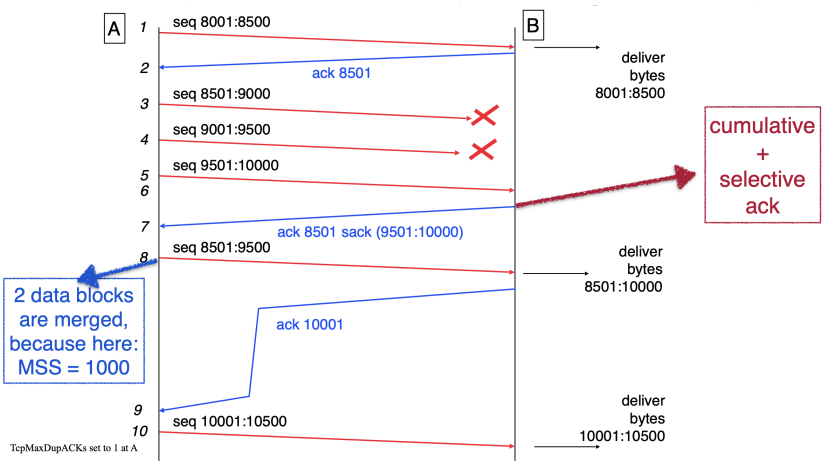
- The applications at A sends that in blocks of 500 bytes at a slow pace. So, TCP initially sends 500-byte segments
- However, the MSS in this example is 1000 bytes. So, TCP may also merge 2 blocks of data in one segment if this data happens to be available at the send buffer of the socket
- B returns an acknowledgement in the ACK field. The ACK Field is *cumulative*, so ACK 85001 means: B is acknowledging all byes up to (excluding) number 8501.
- The out-of-order received data is kept in the buffer until all data is received correctly - this is not visible to the application.



TCP Basic Operations 2: SACK and optimized segmentation (if possible)

In addition to the ACK field, most TCP implementation also use the SACK field (*Selective Acknowledgement*).

- At line 7, B acknowledges all bytes up to 8501 and in the range 9501:10000. Since the set of acknowledged byes is not contiguous, the SACK option is used. It contains up to 3 blocks that are acknowledged in addition to the range described by the ACK field
- At line 8, A detects that the bytes 8501:9500 were lost and re-sends them ASAP without waiting for a timeout. Since the MSS is 1000 bytes, only one packet it sent (optimized segmentation).
- When receiving packet 8, B can deliver bytes 9001:10000 because packet 5 was received and kept in the receive buffer.

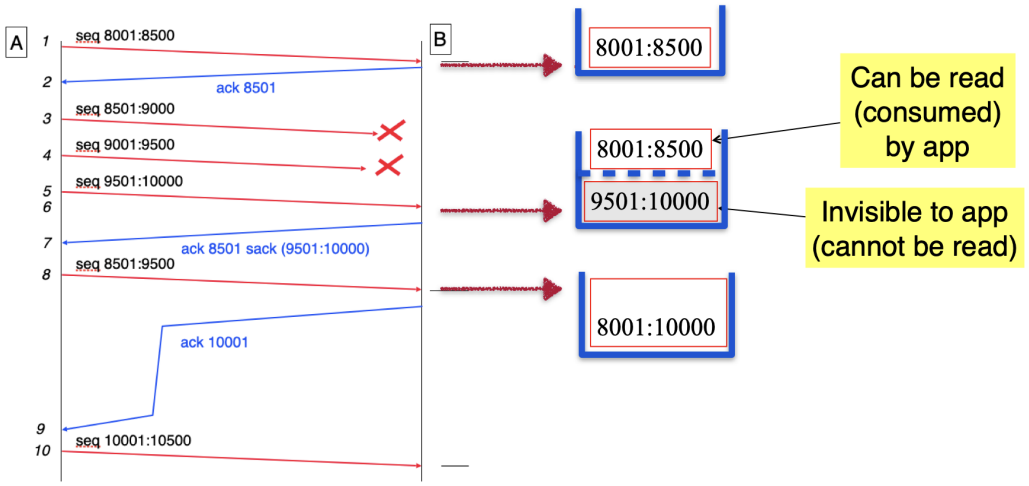


Receive Buffer

TCP receiver uses a ***receive buffer*** = *re-sequencing* buffer to store incoming packets before delivering them to the application.

Why invented?

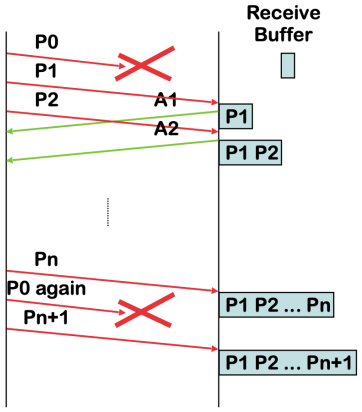
- Application may not be ready to consume/read data
- Packets may need re-sequencing; out-of-order data is stored but is *not visible* to application



TCP uses a sliding window

Why?

- Out-of-order packets may trigger buffer *overflows*
- The sliding window *limits* the number of data



How does the sliding window work?

- Suppose: window size = 4000B; each segment = 1000B.
- Only seq numbers within the window can be sent (and are sent if enough data exist in the socket)
- `lower window edge = smallest non-ack'ed sequence number`
- `upper window edge = lower_edge + window_size`

